Fraudulent Bitcoin Transaction Detection

BT4012 Fraud Analytics — Kaggle Competition Report
Student Name: Liu Yichuan
Student Number: A0292159W
GitHub Repository: https://github.com/lydia372/BT4012_Bitcoin_Fraud_Detection.git

1. Introduction

The increasing adoption of cryptocurrencies has introduced new challenges for financial crime detection. Bitcoin transactions operate on a decentralized network, where funds can be transferred across multiple addresses without traditional financial intermediaries. This creates opportunities for illicit activities, including money laundering and fraudulent transactions, making automated detection an important application of fraud analytics.

This competition focuses on identifying potentially fraudulent Bitcoin transactions using partially labeled transaction data. Each transaction is represented by numerical features and forms part of a directed transaction graph, where edges describe relationships between transactions. The task is formulated as a binary classification problem, with label 1 representing fraudulent transactions and label 0 representing legitimate transactions.

Unlike conventional tabular classification, this problem presents three major challenges. First, the dataset is highly imbalanced, with fraudulent transactions representing approximately 11.7% of labeled training observations. Second, the transaction network contains both labeled and unlabeled nodes, requiring careful integration of graph information. Third, the training and test datasets originate from different time periods, introducing potential temporal distribution shift.

To address these challenges, this project systematically investigates three questions:

1. Can graph-derived features improve transaction-level fraud detection?
2. How does temporal distribution shift affect model generalization?
3. Can graph neural networks complement traditional machine learning models through ensemble learning?

The proposed approach combines tabular machine learning, graph feature engineering, temporal validation, and graph representation learning. The final solution uses a rank-based ensemble of LightGBM and Directed GraphSAGE to integrate complementary transaction-level and network-level information.

2. Methodology

2.1 Data Preprocessing and Feature Engineering

The dataset contains **31,235 labeled training transactions**, including **3,644 illicit transactions (11.7%)**, and **15,329 test transactions**. Each transaction has 165 numerical features, while directed edges represent relationships between transactions. Training observations span time steps 1–28, whereas test observations span steps 36–49.

To capture structural information, we progressively engineered **node degree**, **1-hop and 2-hop neighborhood statistics**, and higher-order graph features. Neighborhood aggregation calculated the mean, standard deviation, minimum, and maximum of neighboring transaction attributes. Because graph coverage was incomplete, we also explored PageRank, graph components, and feature selection to assess whether structural information improved prediction beyond raw transaction features.

Table 1. Graph Feature Ablation

Feature Configuration	Features	ROC-AUC	PR-AUC
Raw features	165	**0.9523**	**0.8802**
Raw + 1-hop	405	0.9314	0.8692
Raw + 2-hop	285	0.9425	0.8705
Raw + 1-hop + 2-hop	525	0.9351	0.8666
Raw + Degree	171	0.9511	0.8791
Raw + 2-hop + Degree	291	0.9486	0.8747

2.2 Model Development and Validation

We initially compared **Logistic Regression, HistGradientBoosting, XGBoost, and LightGBM** using chronological validation. LightGBM was therefore selected as the primary tabular model for subsequent experiments.

To evaluate temporal generalization, we constructed three chronological validation settings:

- **W1:** Train ≤14; validate 15–28.
- **W2:** Train ≤21; validate 22–35.
- **W3:** Train ≤14; validate 22–35.

These settings allowed us to examine sensitivity to different training histories and validation periods while avoiding the use of future labeled observations during model training.

Adversarial validation and feature-distribution analysis indicated substantial differences between historical and future observations. We consequently explored **Kolmogorov–Smirnov (KS)**

feature filtering to remove highly shifted features and **exponential time-decay weighting** to emphasize recent transactions. The decay coefficient was tuned experimentally, with λ = 0.02 selected based on the strongest observed performance: $w_i = exp[-0.02(T - t_i)]$

where (t_i) denotes a transaction's time step and (T) is the latest training time step.

Graph coverage analysis revealed that 22.9% of graph nodes lacked observable features or timestamps, while approximately 39.1% of test-node neighbor connections involved nodes without usable information. This motivated an investigation into GraphSAGE robustness under incomplete graph observations. Dropout-based regularization was explored, and the final GraphSAGE architecture applied 30% hidden-layer dropout to reduce overfitting. However, hidden-layer dropout does not directly address missing graph connections.

ROC-AUC was the primary evaluation metric, while **PR-AUC** was used to assess performance under class imbalance.

2.3 Model Exploration and Final Strategy

Beyond traditional classifiers, we investigated **Vanilla GraphSAGE** and **Directed GraphSAGE** to learn graph representations directly. Vanilla GraphSAGE aggregates neighborhood information, whereas Directed GraphSAGE distinguishes incoming and outgoing transaction relationships.

Both models were evaluated against LightGBM under chronological validation. We also explored rank-based ensembles to determine whether their predictions contained complementary information.

The final submission combined normalized prediction ranks from LightGBM and Directed GraphSAGE: $[S = 0.75 * LGBM + 0.25 * DirectedSAGE]$

This configuration was selected for its Public Leaderboard performance, while its weaker chronological validation results were considered an important limitation.

3. Results and Discussion

Table 2. Chronological Validation Results (ROC-AUC)

Model	W1	W2	W3	Public Score
LightGBM	**0.9656**	**0.9010**	0.8698	0.94261
Vanilla GraphSAGE	0.9392	0.8825	0.8536	0.95963
Directed GraphSAGE	0.9198	0.8363	0.8196	0.93195

LGBM + Directed (75/25)	0.9628	0.8905	0.8647	**0.96095**
LGBM + Vanilla (80/20)	0.9652	0.9010	**0.8718**	0.96050

Note: Leaderboard submissions represent different experimental configurations and should not be interpreted as a controlled ablation.

Performance varied substantially across chronological settings. LightGBM's ROC-AUC decreased from **0.9656 in W1** to **0.8698 in W3**, demonstrating sensitivity to the evaluation period and available training history.

KS-based filtering improved performance in selected validation configurations, suggesting that excluding highly shifted features can sometimes improve robustness. Time-decay weighting was also explored to prioritize recent transaction patterns.

Standalone GraphSAGE models generally underperformed LightGBM. Although the LightGBM–Vanilla GraphSAGE ensemble slightly improved W3 performance, the gains were limited and inconsistent across validation windows.

4. Conclusion and Reflection

This project explored graph feature engineering, temporal adaptation, and graph neural networks for Bitcoin fraud detection, achieving a final Public ROC-AUC of **0.96095** through a LightGBM–Directed GraphSAGE ensemble. However, despite extensive experimentation, more complex approaches did not consistently outperform the original LightGBM baseline in chronological validation.

A key unresolved question is why additional graph information failed to deliver reliable improvements. The original transaction features may already capture strong predictive signals, while neighborhood aggregation could introduce noise. Incomplete graph coverage may have further limited GraphSAGE's effectiveness. Another limitation lies in the temporal validation design: W1–W3 differed in training history and prediction horizon, making it difficult to distinguish distribution shift from differences in available information. Although exponential time-decay weighting was incorporated, its effectiveness across future periods remains uncertain.

Finally, the ensemble's improved Public Leaderboard performance contrasted with its weaker results across all three local validation windows. This highlighted the risk of relying on leaderboard feedback when selecting models. If revisiting this project, I would prioritize more consistent expanding-window validation, improved graph coverage, and stronger evidence of complementary predictive information before adopting complex ensembles.